



Communauté Française de Belgique

Institut des Carrières Commerciales

Ville de Bruxelles

Waterside

Quai de Willebroeck, 33

1000 Bruxelles



Documentation

API et Open Data

Tarik EL MAHTOUCI

2025-2026

Table des matières

1. Introduction.....	2
2. Requêtes GET.....	2
3. Requêtes POST.....	2
4. Requêtes PUT.....	3
5. Requêtes DELETE.....	3
6. Conclusion.....	3

1. Introduction

Cette documentation décrit l'API de BikeShare, plateforme de location de vélos entre particuliers. L'API expose les fonctionnalités clés de la plateforme : gestion des utilisateurs, des vélos, des réservations, des paiements et des évaluations. Elle permet aux développeurs d'intégrer ou d'étendre les fonctionnalités de BikeShare au sein d'applications tierces.

L'API est documentée et accessible via Swagger (springdoc-openapi), un outil qui fournit une interface interactive pour explorer les endpoints, tester les requêtes et visualiser les réponses. La documentation OpenAPI est exposée sur [/v3/api-docs](#) et l'interface Swagger UI sur [/swagger-ui.html](#). Chaque endpoint est détaillé avec ses paramètres, le type de réponse attendu et les codes d'état HTTP possibles.

2. Requêtes GET

Méthode	Endpoint	Description	Paramètres / Réponses
GET	/api/admin/payments/{id}	Obtenir un paiement spécifique par son ID.	path: id (integer) — 200 : OK
GET	/api/admin/features	Récupérer toutes les caractéristiques.	200 : OK
GET	/api/admin/categories	Récupérer toutes les catégories disponibles.	200 : OK
GET	/api/admin/bikes/{id}	Obtenir un vélo par ID.	path: id (integer) — 200 : OK
GET	/api/admin/bikes/pending	Obtenir tous les vélos en attente d'approbation.	200 : OK
GET	/api/admin/bikes/online	Obtenir tous les vélos en ligne.	200 : OK
GET	/api/notifications/filter	Filtrer les notifications selon des critères.	query params — 200 : OK
GET	/api/messages/reservation/{reservationId}	Obtenir les messages échangés pour une réservation.	path: reservationId — 200 : OK
GET	/api/unavailable-dates	Obtenir les dates d'indisponibilité d'un vélo.	query: bikeId (integer) — 200 : OK
GET	/api/reservations	Obtenir les réservations de l'utilisateur courant.	200 : OK
GET	/api/getOwnerReservations	Obtenir les réservations pour les vélos du propriétaire.	query: bikeIds (array) — 200 : OK
GET	/api/getRenterReservations	Obtenir les réservations selon l'état pour le locataire.	query: statuses (array) — 200 : OK
GET	/api/download-contract/{reservationId}	Télécharger le contrat de réservation pour un vélo.	path: reservationId — 200 : OK (PDF)

3. Requêtes POST

Méthode	Endpoint	Description	Paramètres / Réponses
POST	/api/review	Créer une évaluation pour un vélo.	body: EvaluationDTO — 200 : OK
POST	/api/payments/create-payment-intent	Créer une intention de paiement.	body: { amount, currency, reservationId } — 200 : OK
POST	/api/payments/confirm	Confirmer une réservation après paiement.	body: { reservationId, paymentIntentId, insurance } — 200 : OK
POST	/api/password/forgot	Envoyer un email de réinitialisation de mot de passe.	query: email (string) — 200 : OK
POST	/api/notifications/complaint	Envoyer une plainte à l'utilisateur concerné.	body: NotificationDTO — 200 : OK

Méthod e	Endpoint	Description	Paramètres / Réponses
POST	/api/bikes/search/category	Rechercher des vélos par catégorie.	body: { category: string } — 200 : OK
POST	/api/cancelReservation/{id}	Annuler une réservation et calculer les remboursements.	path: id (integer) — 200 : OK
POST	/api/admin/features	Ajouter une nouvelle caractéristique.	body: Feature — 200 : OK
POST	/api/admin/equipments	Ajouter un nouvel équipement avec une icône.	body: { equipment, iconFile } — 200 : OK
POST	/api/admin/categories	Ajouter une nouvelle catégorie.	body: Category — 200 : OK
POST	/api/admin/bikes/approve/{id}	Approuver un vélo pour le rendre disponible en ligne.	path: id (integer) — 200 : OK

4. Requêtes PUT

Méthod e	Endpoint	Description	Paramètres / Réponses
PUT	/api/admin/payments/{id}	Mettre à jour le statut d'un paiement.	path: id ; body: PaymentDTO — 200 : OK
PUT	/api/admin/users/{userId}/permissions	Mettre à jour les permissions d'un utilisateur.	path: userId ; body: { permissions } — 200 : OK

5. Requêtes DELETE

Méthod e	Endpoint	Description	Paramètres / Réponses
DELETE	/api/admin/payments/{id}	Supprimer un paiement spécifique.	path: id (integer) — 200 : OK
DELETE	/api/admin/users/{userId}	Supprimer un utilisateur de la plateforme.	path: userId (integer) — 200 : OK
DELETE	/api/admin/users/{userId}/documents/{documentType}	Supprimer un document spécifique d'un utilisateur.	path: userId, documentType — 200 : OK
DELETE	/api/admin/evaluations/{id}	Supprimer une évaluation par son ID.	path: id (integer) — 200 : OK
DELETE	/api/users/{id}	Supprimer un utilisateur de la plateforme.	path: id (integer) — 200 : OK

6. Conclusion

L'API de BikeShare offre une interface complète et flexible pour interagir avec les principales fonctionnalités de la plateforme de location de vélos. Qu'il s'agisse de gérer les utilisateurs, de publier et consulter des annonces de vélos, de traiter les réservations ou de gérer les paiements, cette API permet une intégration fluide dans divers systèmes tiers. Grâce à Swagger, les développeurs peuvent facilement explorer et tester l'API afin de garantir une implémentation efficace et fiable.